

Student name: Jordan Elkin

Student ID: 226621873

SIT225: Data Capture Technologies

Activity 8.1: Using smartphone to capture sensor data

The **Arduino IoT Remote** phone application lets you control and monitor all of your dashboards in the Arduino Cloud. With the app, you can also access your phone's internal sensors such as GPS data, light sensor, IMU and more (depending on what phone you have).

The phone's sensor data is automatically stored in Cloud variables, which you can also synchronize with other Things such as custom thing in Python board. This means your phone can become a part of your IoT system, acting as another node in your network.

In this activity, you will enable your smartphone to work as a custom device (like an Arduino board) and connect to your smartphone sensors such as accelerometers and GPS and streaming data to Arduino IoT Cloud dashboard.

Hardware Required

Your smartphone – compatible Android or iPhone

NOTE: *The IoT Remote app requires iOS 12.4 or later for iOS the version. If you are using Android, version 8.0 or later is required. Make sure the iOS or Android version on your device is up to date before downloading the app.*

Software Required

Android / iOS smart phone.

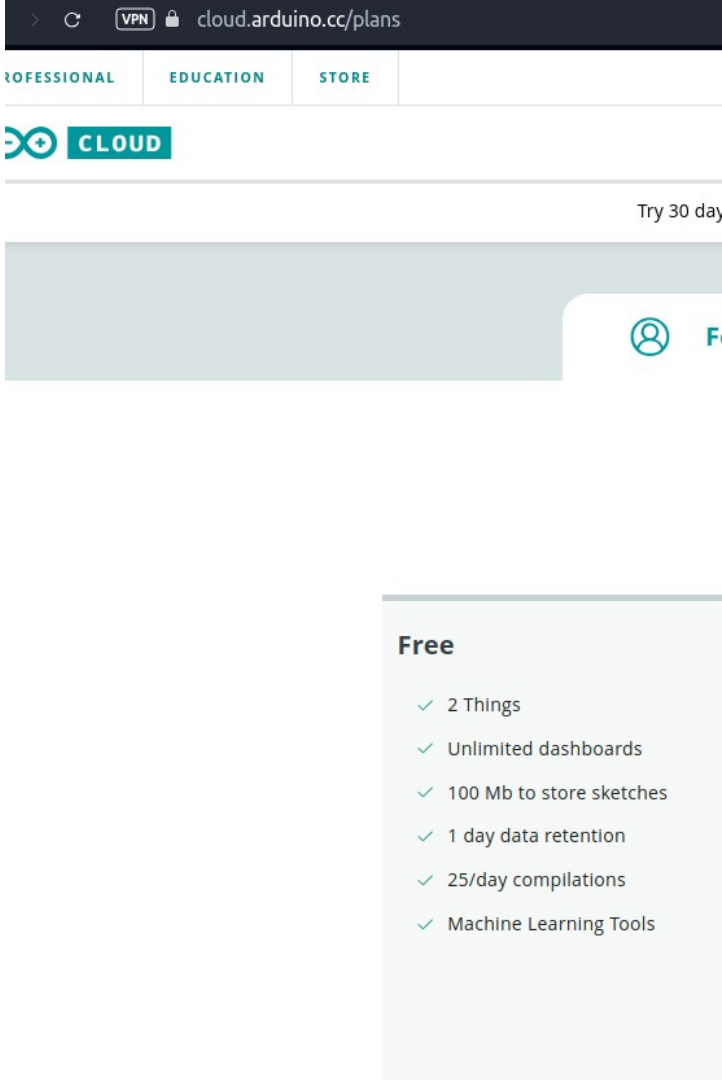
Arduino account

Arduino IoT Remote App (App Store or Google Play)

Python 3 (for custom Python Thing)

Steps

Step	Action
1	Install App: To use the Arduino IoT Remote app, visit Google Play / App Store and search for "Arduino IoT Remote". After installing the app, you will need to log in to your Arduino account. After you login, you will discover all your dashboards (if you have any), in the main menu. Based on the app version, home screen may vary. There will be 3 tabs at the bottom – Dashboards, Devices and Activity. You can follow the tutorial (https://docs.arduino.cc/arduino-cloud/iot-remote-app/getting-started).
2	Add device: Tap into the Devices tab. You will be able to create a new device. Alternatively, you can your profile (top right corner), in the settings section, you will see "Phone as device" which you can turn ON if it is OFF. There, you can select sensors in your smartphone such as accelerometer linear, accelerometer x/y/z and GPS among others. Note: A free account is enough for this experiment. If you are asked to upgrade your account, you can remove all other Things from your Arduino IoT Cloud account since the Free account allows at most 2 Things to configure, see below image.

	 The add device wizard will allow you to setup your sensor and also create a dashboard which you can see in your smartphone app. If you login to Arduino IoT Cloud in web browser, you can see the dashboard for your smartphone is already created.
3	Keep your smartphone screen ON for a while: Keep data coming through your smartphone for 10-15 minutes. During this time, keep moving your smartphone in a pattern so the accelerometer data can be analysed to discover the pattern. You can download data from the Ardiuno Cloud dashboard page by clicking on a download icon at top right corner which shows – Download historic data. A data download link will be sent to your account email from there you can download data. Question: Take a screenshot of your Ardiuno Cloud Dashboard where

smartphone data is streaming and paste it here.

Answer:



4

Plot accelerometer data:

The zipped data file you downloaded from the cloud contains separate files per variable including accelerometer_linear, accelerometer_x, accelerometer_y, accelerometer_z and Gps. Each file has 2 columns – time and value.

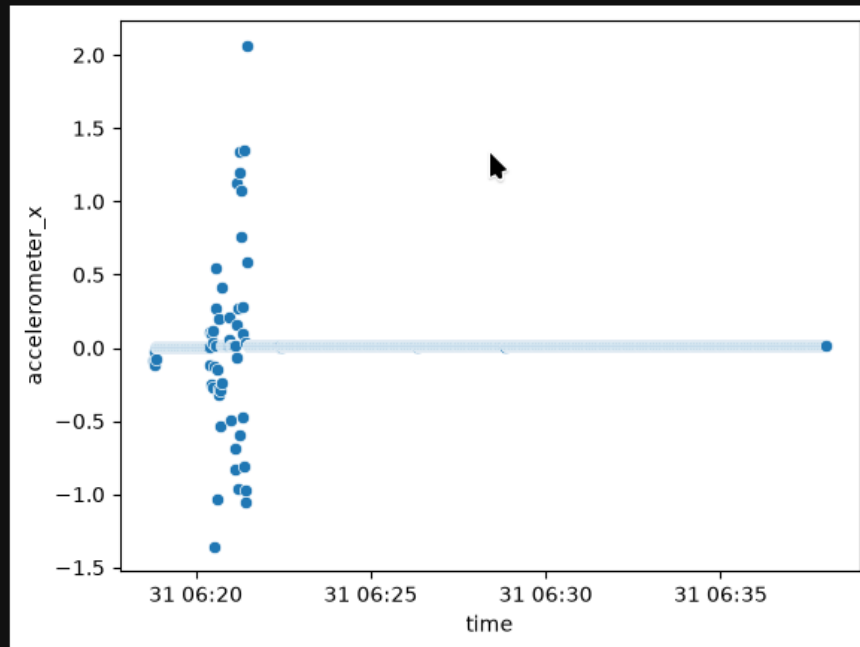
Question: Open Jupyter Notebook by using command line, go to the data folder and write command (`$ jupyter lab`). Using Pandas, read CSV file and fetch the data column for accelerometer_x and plot it using Python plotting library (matplotlib or any other convenient for you). Repeat the plotting process for accelerometer y and z to have 3 separate graphs. Now create a fourth graph with all 3 variables x, y and z. Screenshot the 4 graphs and paste here.

Answer:

Accelerometer_X vs Time

```
[78]: sns.scatterplot(data=accel_x,x='time',y='accelerometer_x',legend='auto')
```

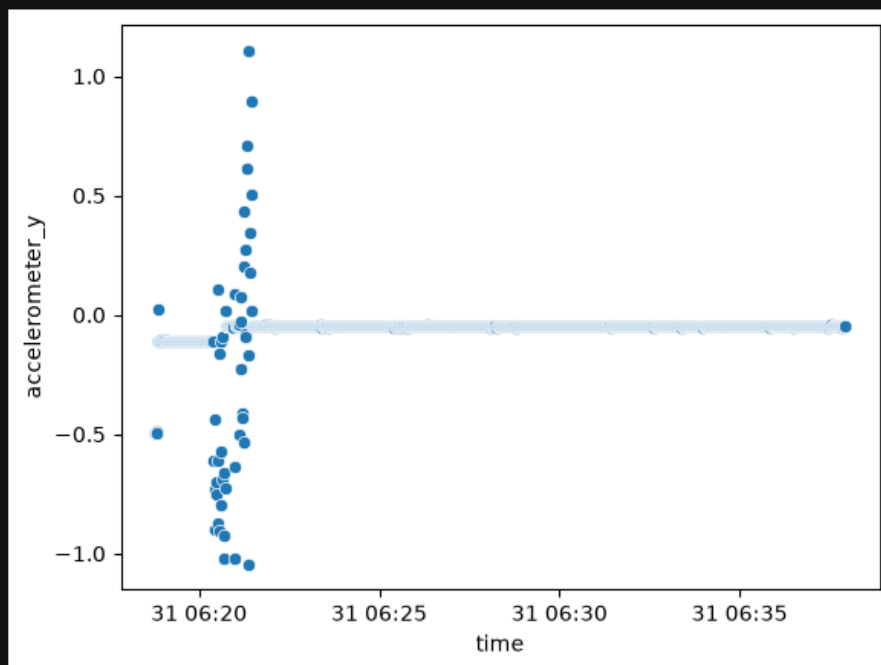
```
[78]: <Axes: xlabel='time', ylabel='accelerometer_x'>
```



Acceleromter_Y vs Time

```
[79]: sns.scatterplot(data=accel_y,x='time',y='accelerometer_y',legend='auto')
```

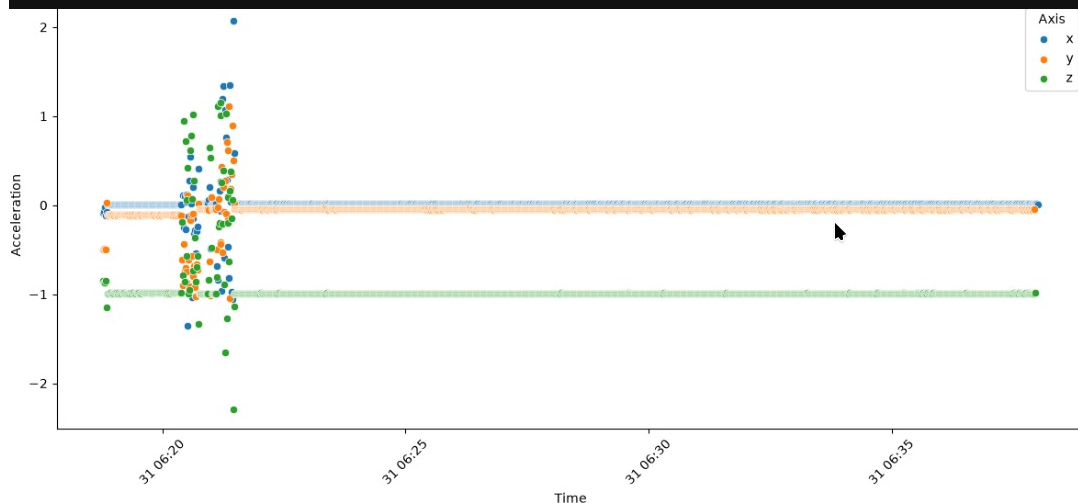
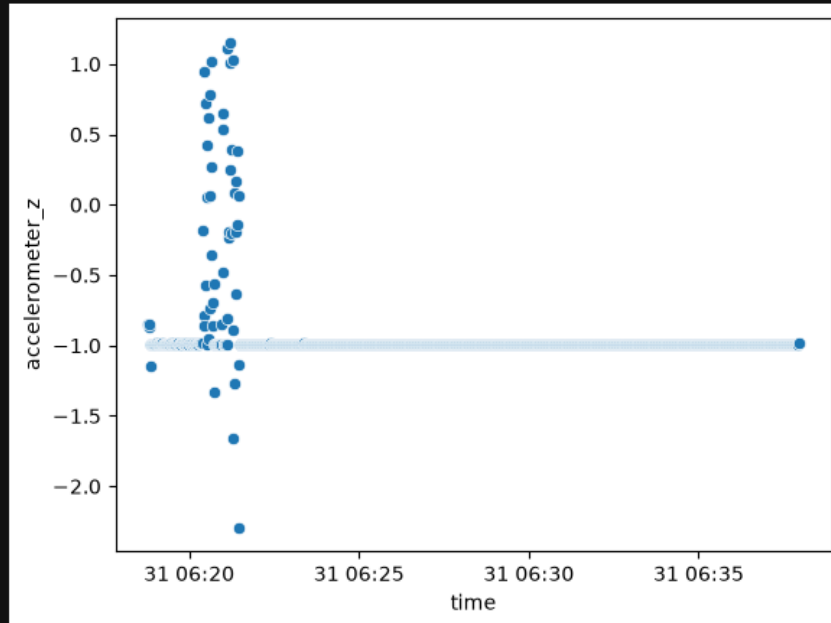
```
[79]: <Axes: xlabel='time', ylabel='accelerometer_y'>
```



Accelerometer_Z vs Time

```
[80]: sns.scatterplot(data=accel_z, x='time', y='accelerometer_z', legend='auto')
```

```
[80]: <Axes: xlabel='time', ylabel='accelerometer_z'>
```



5

Question: Analyse accelerometer variables to find any repeating pattern. Remember that you were repeatedly moving your phone in a single pattern which should be manifested in the graphs. Justify your answer.

Answer:

The main repeating pattern can be seen at the beginning of the plot where the x,y,z variables are fluctuating at the same rate before becoming stationary for the rest of the collection period.

--	--

Activity 8.2: Receive smartphone sensor data from Python script

You can connect anything to Arduino Cloud including a wide range of compatible Arduino boards such as Arduino Nano 33 IoT or a third-party device that speaks Python. In activity 3.2, you have configured custom Python board and created a cloud variable that was synced to your Arduino Thing such as DHT22 sensor variables.

In this activity, you will need to synchronise smartphone's accelerometer x, y and z variables to Python script. If you can recall, you have already done a similar function in Activity 3.2.

Steps:

Step	Action
1	Configure Python board in Arduino Cloud and create a Thing where define 3 variables at a time and sync to corresponding accelerometer variable of smartphone Thing.
2	Write Python script to keep listening to data from the 3 variables to come through. You may need to create 3 call-back functions – a single function per variable (x, y and z).
3	Question: Keep storing each variable data in a separate file. Append each value with a timestamp so each data reading forms a comma separated line - <timestamp>, <data-value>. New data is written in a separate line. Keep storing them in a CSV file, where there will be 3 separate files. Screenshot your Python script here and screenshot the files opened side-by-side you have created and paste it here. Answer:

```

1 import sys
2 import traceback
3 import random
4 from arduino_iot_cloud import ArduinoCloudClient
5 import asyncio
6 from datetime import datetime
7 import time
8
9 accel_x, accel_y, accel_z = 0.0, 0.0, 0.0
10
11 rec_accel_x = False
12 rec_accel_y = False
13 rec_accel_z = False
14
15
16 # Callback function on change events.
17
18 def on_x_changed(client, value):
19     global accel_x
20     accel_x = value
21     global rec_accel_x
22     accel_x_csv = open("accel_x.csv", 'a+')
23     print(f"New x value: {value}")
24     accel_x_csv.write(f"{datetime.now().strftime('%H:%M:%S')},{value}\n")
25     accel_x_csv.close()
26     rec_accel_x = True
27     save_combined()
28
29 def on_y_changed(client, value):
30     global accel_y
31     accel_y = value
32     global rec_accel_y
33     accel_y_csv = open("accel_y.csv", 'a+')
34     print(f"New y value: {value}")
35     accel_y_csv.write(f"{datetime.now().strftime('%H:%M:%S')},{value}\n")
36     accel_y_csv.close()
37     rec_accel_y = True
38     save_combined()
39
40 def on_z_changed(client, value):
41     global accel_z
42     global rec_accel_z
43     accel_z = value
44     accel_z_csv = open("accel_z.csv", 'a+')
45     print(f"New z value: {value}")
46     accel_z_csv.write(f"{datetime.now().strftime('%H:%M:%S')},{value}\n")
47     accel_z_csv.close()
48     rec_accel_z = True
49     save_combined()
50
51 New z value: -0.9905242919921875
52 New x value: 0.0164031982421875
53 New y value: -0.0420684814453125
54 New z value: -0.9880218505859375
55 New x value: 0.0182952880859375
56 New y value: -0.0438690185546875
57 New z value: -0.989776611328125
58 New x value: 0.0186920166015625
59 New y value: -0.0434417724609375
60 New z value: -0.9892120361328125
61 New x value: 0.019256591796875
62 New y value: -0.044830322265625
63 New z value: -0.98907470703125

```

	A	B	C	D
1	18:32:25	0.0178375244140625		
2	18:32:28	0.01751708984375		
3	18:32:31	0.01629638671875		
4	18:32:35	0.01666259765625		
5	18:32:38	0.0151824951171875		
6	18:32:42	0.01788330078125		
7	18:32:45	0.01837158203125		
8	18:32:48	0.0170440673828125		
9	18:32:51	0.0167694091796875		
10	18:32:54	0.0170135498046875		
11	18:32:57	0.0183868408203125		
12	18:32:59	0.0153656005859375		
13	18:33:02	0.0170745849609375		
14	18:33:05	0.01611328125		
15	18:33:08	0.016204833984375		
16	18:33:12	0.016998291015625		
17	18:33:15	0.01702880859375		
18	18:33:18	0.016632080078125		
19	18:33:21	0.0170135498046875		
20	18:33:24	0.0169219970703125		
21	18:33:27	0.0175323486328125		
22	18:33:30	0.0174560546875		
23	18:33:33	0.0167083740234375		
24	18:33:36	0.016571044921875		
25	18:33:39	0.0158843994140625		
26	18:33:42	0.0152587890625		
27	18:33:46	0.01605224609375		
28	18:33:49	0.0173492431640625		
29	18:33:52	0.0177154541015625		
30	18:33:55	0.0173187255859375		
31	18:33:58	0.016632080078125		
32	18:34:01	0.01629638671875		
33	18:34:04	0.0142669677734375		

18:32:26	-0.0507659912109375			
18:32:29	-0.049072265625			
18:32:33	-0.050018310546875			
18:32:36	-0.04815673828125			
18:32:39	-0.0488433837890625			
18:32:43	-0.0485076904296875			
18:32:46	-0.049346923828125			
18:32:49	-0.0500030517578125			
18:32:52	-0.049591064453125			
18:32:55	-0.0484771728515625			
18:33:00	-0.048248291015625			
18:33:03	-0.04888916015625			
18:33:06	-0.0498199462890625			
18:33:09	-0.04986572265625			
18:33:13	-0.0499725341796875			
18:33:16	-0.0496063232421875			
18:33:19	-0.050079345703125			
18:33:22	-0.04901123046875			
18:33:25	-0.049774169921875			
18:33:28	-0.0495758056640625			
18:33:31	-0.049224853515625			
18:33:34	-0.05059814453125			
18:33:37	-0.051177978515625			
18:33:40	-0.0477294921875			
18:33:43	-0.049530029296875			
18:33:47	-0.0494232177734375			
18:33:50	-0.0501708984375			
18:33:53	-0.0495758056640625			
18:33:56	-0.05023193359375			
18:33:59	-0.0509185791015625			
18:34:02	-0.0502166748046875			
18:34:05	-0.0472259521484375			
18:34:08	-0.267822265625			

		A	B	C
1	18:32:24	-0.990875244140625		
2	18:32:27	-0.989105224609375		
3	18:32:30	-0.988235473632813		
4	18:32:34	-0.988616943359375		
5	18:32:37	-0.990478515625		
6	18:32:40	-0.988754272460938		
7	18:32:44	-0.98828125		
8	18:32:47	-0.988540649414063		
9	18:32:50	-0.989303588867188		
10	18:32:53	-0.9898681640625		
11	18:32:56	-0.988754272460938		
12	18:32:58	-0.989288330078125		
13	18:33:01	-0.989013671875		
14	18:33:04	-0.988296508789063		
15	18:33:07	-0.988327026367188		
16	18:33:10	-0.989486694335938		
17	18:33:14	-0.989654541015625		
18	18:33:17	-0.989501953125		
19	18:33:20	-0.988739013671875		
20	18:33:23	-0.988174438476563		
21	18:33:26	-0.989227294921875		
22	18:33:29	-0.987411499023438		
23	18:33:32	-0.9881591796875		
24	18:33:35	-0.989242553710938		
25	18:33:38	-0.987350463867188		
26	18:33:41	-0.989715576171875		
27	18:33:44	-0.987823486328125		
28	18:33:48	-0.988296508789063		
29	18:33:51	-0.9891357421875		
30	18:33:54	-0.987640380859375		
31	18:33:57	-0.98919677734375		
32	18:34:00	-0.988037109375		
33	18:34:03	-0.988800048828125		
4	Question: Now manage 3 variable data so they can be stored in a single CSV file where each line consists of comma separated sensor values with a timestamp - <timestamp>, <x>, <y>, <z>. Store data once			

you gather 3 variables and repeat the process. Screenshot your Python script here and screenshot the file you have created opened and paste it here.

Answer:

	A	B	C	D	E	
1	Time	X	Y	Z		
2	18:50:40	0.0185546875	-0.0426025390625	-0.98828125		
3	18:50:49	-0.377517700195313	0.659133911132813	-0.307525634765625		
4	18:50:52	-0.19952392578125	0.454727172851563	-0.068939208984375		
5	18:50:55	-0.177780151367188	0.849472045898438	-0.241012573242188		
6	18:56:54	-0.0338897705078125	-0.700927734375	-0.68719482421875		
7	18:56:58	0.0162200927734375	-0.678131103515625	-0.71826171875		
8	18:57:01	0.0212860107421875	-0.664962768554688	-0.7996826171875		
9	18:57:04	0.500946044921875	-0.0264434814453125	-0.55755615234375		
10	18:57:07	0.017669677734375	-0.0459747314453125	-0.991287231445313		
11	18:57:10	0.01568603515625	-0.043487548828125	-0.99029541015625		
12	18:57:13	0.01800537109375	-0.044342041015625	-0.990631103515625		
13	18:57:16	0.016021728515625	-0.044677734375	-0.991683959960938		
14	18:57:19	0.020843505859375	-0.0439453125	-0.991317749023438		
15	18:57:22	0.01934814453125	-0.0442047119140625	-0.990798950195313		
16	18:57:25	0.017303466796875	-0.0452423095703125	-0.992599487304688		
17	18:57:28	0.016265869140625	-0.0449371337890625	-0.990875244140625		
18	18:57:32	0.0169219970703125	-0.0447845458984375	-0.991897583007813		
19	18:57:35	0.0166778564453125	-0.04327392578125	-0.991683959960938		
20	18:57:38	0.0179901123046875	-0.0468292236328125	-0.990875244140625		
21	18:57:41	0.0176544189453125	-0.0463714599609375	-0.992050170898438		
22	18:57:44	0.0180206298828125	-0.045013427734375	-0.9925537109375		
23	18:57:47	0.0169525146484375	-0.04571533203125	-0.9903564453125		
24	18:57:50	0.0152130126953125	-0.0443878173828125	-0.993240356445313		
25	18:57:53	0.0169677734375	-0.0429229736328125	-0.991302490234375		
26	18:57:56	0.016754150390625	-0.0452880859375	-0.99160921875		

```
def on_y_changed(client,value):
    global accel_y
    accel_y = value
    global rec_accel_y
    accel_y_csv = open("accel_y.csv",'a+')
    print(f"New y value: {value}")
    accel_y_csv.write(f"{datetime.now().strftime('%H:%M:%S')},{value}\n")
    accel_y_csv.close()
    rec_accel_y = True
    save_combined()

def on_z_changed(client,value):
    global accel_z
    global rec_accel_z
    accel_z = value
    accel_z_csv = open("accel_z.csv",'a+')
    print(f"New z value: {value}")
    accel_z_csv.write(f"{datetime.now().strftime('%H:%M:%S')},{value}\n")
    accel_z_csv.close()
    rec_accel_z = True
    save_combined()

def save_combined():
    global rec_accel_x, rec_accel_y, rec_accel_z
    if rec_accel_x and rec_accel_y and rec_accel_z:
        acceleration_combined = open("combined.csv",'a+')
        acceleration_combined.write(f"{datetime.now().strftime('%H:%M:%S')},{accel_x},{accel_y},{accel_z}\n")
        acceleration_combined.close()
        rec_accel_x = False
        rec_accel_y = False
        rec_accel_z = False
    else:
        return

def main():
```

Activity Week 8

```
In [77]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

accel_linear = pd.read_csv('data/iPhone Thing-Accelerometer_Linear.csv')
accel_x = pd.read_csv('data/iPhone Thing-Accelerometer_X.csv')
accel_x = accel_x.rename(columns={'value': 'accelerometer_x'})

accel_y = pd.read_csv('data/iPhone Thing-Accelerometer_Y.csv')
accel_y = accel_y.rename(columns={'value': 'accelerometer_y'})

accel_z = pd.read_csv('data/iPhone Thing-Accelerometer_Z.csv')
accel_z = accel_z.rename(columns={'value': 'accelerometer_z'})

gps = pd.read_csv('data/iPhone Thing-Gps.csv')

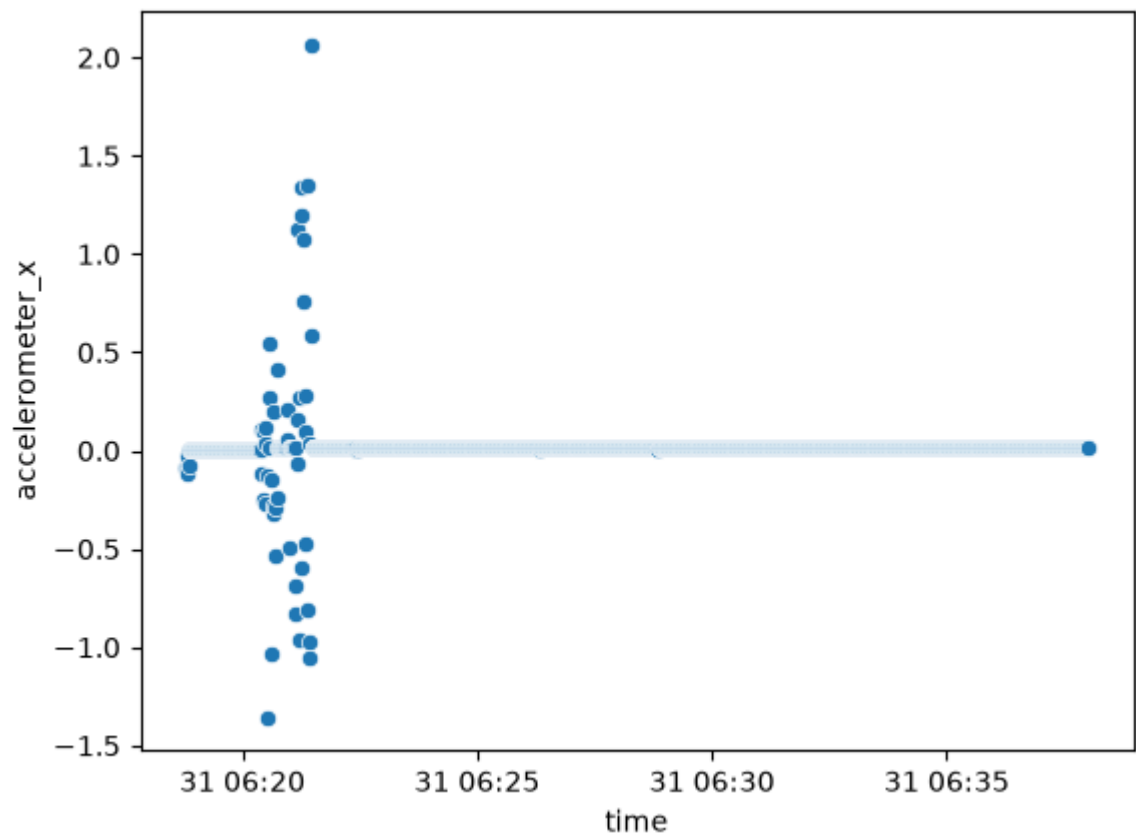
accel_x["time"] = pd.to_datetime(accel_x["time"])
accel_y["time"] = pd.to_datetime(accel_y["time"])
accel_z["time"] = pd.to_datetime(accel_z["time"])

data_full = [accel_linear, accel_x, accel_y, accel_z, gps]
```

Accelerometer_X vs Time

```
In [78]: sns.scatterplot(data=accel_x, x='time', y='accelerometer_x', legend='auto')
```

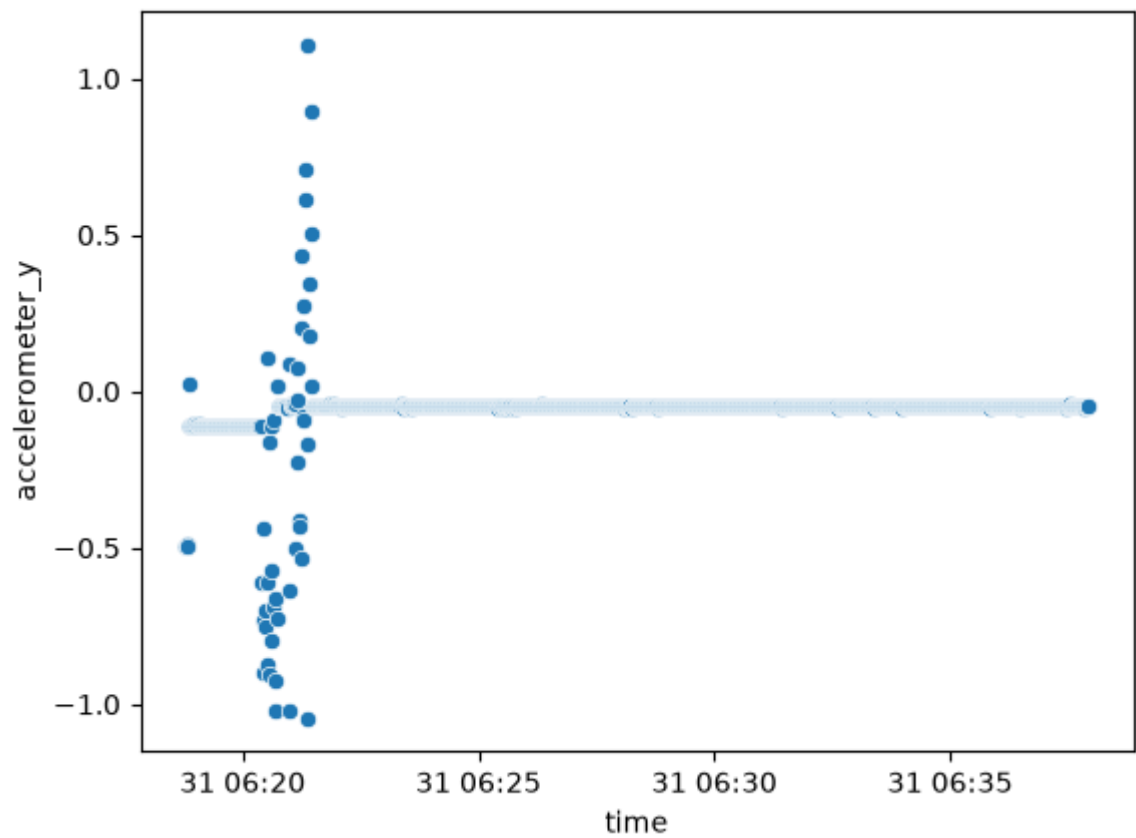
```
Out[78]: <Axes: xlabel='time', ylabel='accelerometer_x'>
```



Acceleromter_Y vs Time

```
In [79]: sns.scatterplot(data=accel_y,x='time',y='accelerometer_y',legend='auto')
```

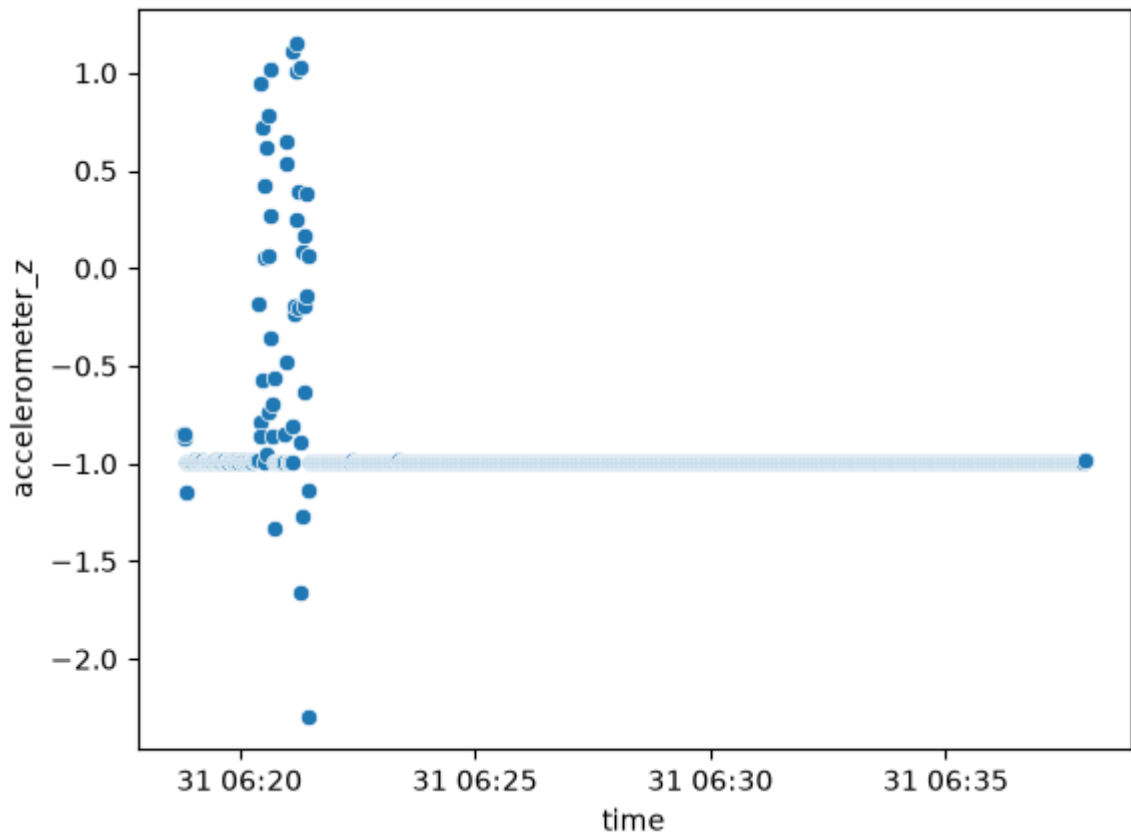
```
Out[79]: <Axes: xlabel='time', ylabel='accelerometer_y'>
```



Accelerometer_Z vs Time

```
In [80]: sns.scatterplot(data=accel_z, x='time', y='accelerometer_z', legend='auto')
```

```
Out[80]: <Axes: xlabel='time', ylabel='accelerometer_z'>
```



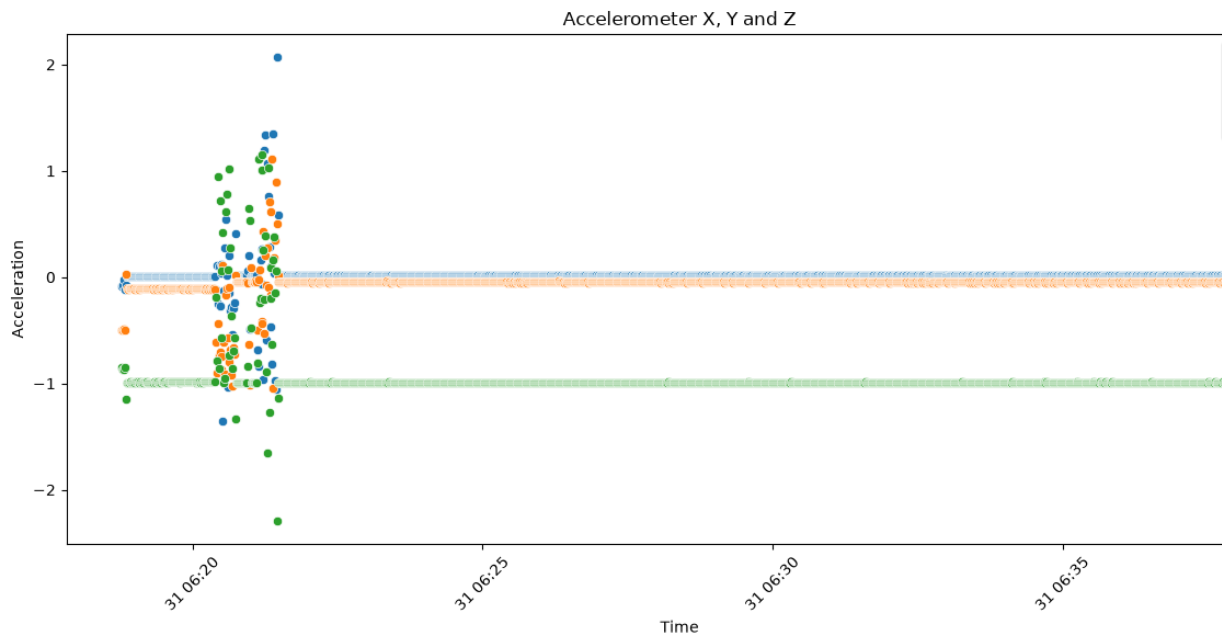
Accelerometer X,Y, Z vs Time

```
In [84]: plt.figure(figsize=(12, 6))

sns.scatterplot(data=accel_x, x='time', y='accelerometer_x', label='x')
sns.scatterplot(data=accel_y, x='time', y='accelerometer_y', label='y')
sns.scatterplot(data=accel_z, x='time', y='accelerometer_z', label='z')

plt.title("Accelerometer X, Y and Z")
plt.xlabel("Time")
plt.ylabel("Acceleration")
plt.xticks(rotation=45)
plt.legend(title="Axis")

plt.tight_layout()
plt.show()
```



Accelerometer X, Y,Z 3 D

```
In [81]: min_len = min(len(accel_x['accelerometer_x']), len(accel_y['accelerometer_

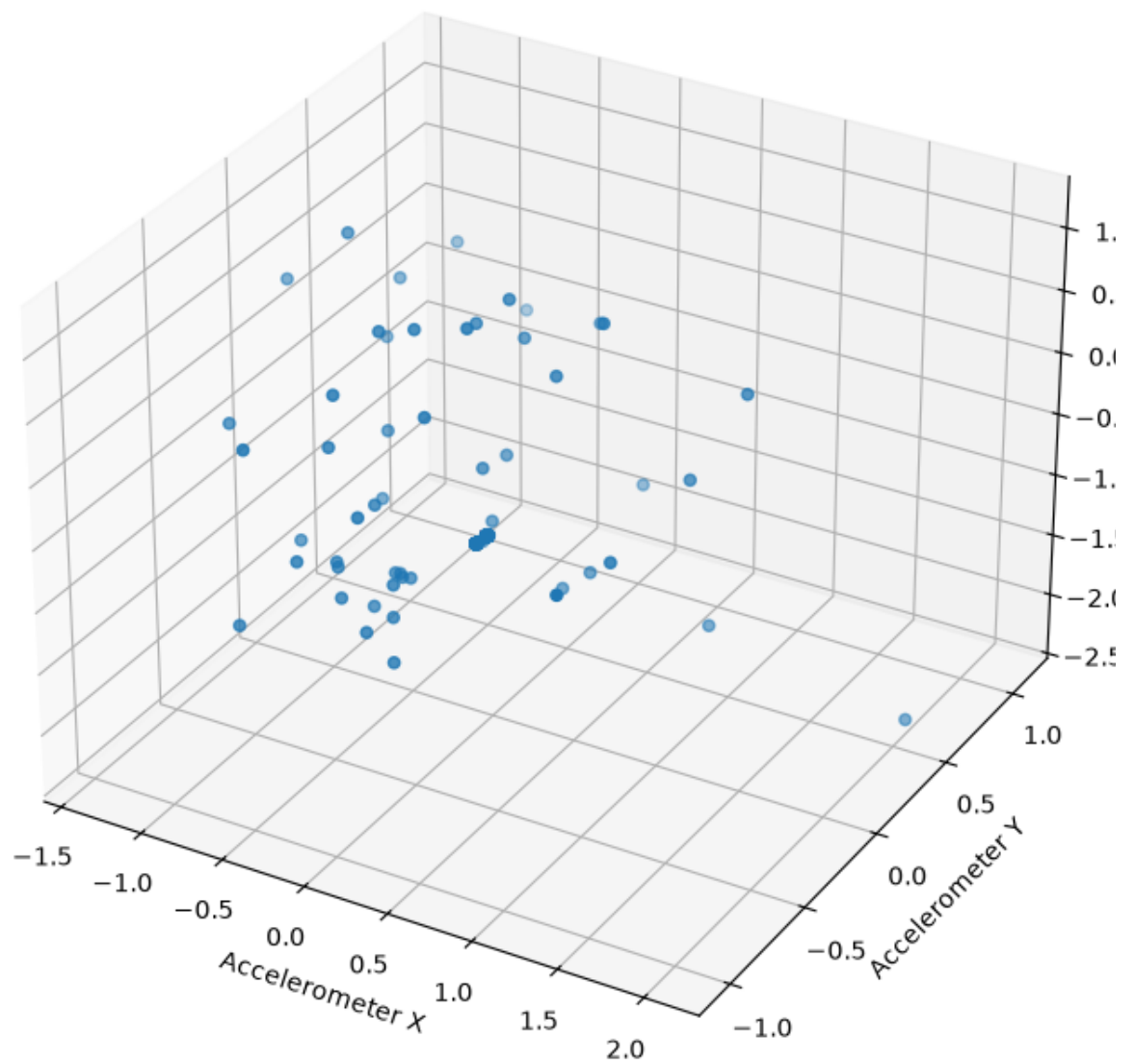
fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(projection='3d')

scatter = ax.scatter3D(accel_x['accelerometer_x'][:min_len], accel_y['acc

# Labels for all three dimensions
ax.set_xlabel('Accelerometer X')
ax.set_ylabel('Accelerometer Y')
ax.set_zlabel('Accelerometer Z')
ax.set_title('Accelerometer X,Y,Z')

plt.tight_layout()
plt.show()
```

Accelerometer X,Y,Z



Python Board Script

In [4]:

```
['d862b630-af90-4581-90f7-693bfa8df8e8\n', 's!CpioLctVoAW3H?ZLvU69jx?\n']  
main() function
```

```

-----
RuntimeError                                Traceback (most recent call last)
Cell In[4], line 64
     62     except:
     63         exc_type, exc_value, exc_traceback = sys.exc_info()
--> 64         traceback.print_tb(exc_traceback, file=print)

Cell In[4], line 56, in main()
     54
     55     # start cloud client
--> 56     client.start()

File ~/Uni/Data Capture Technologies/sit225/lib/python3.14/site-packages/
arduino_iot_cloud/ucloud.py:438, in ArduinoCloudClient.start(self, interval,
backoff)
     437 if self.async_mode:
--> 438     asyncio.run(self.run(interval, backoff))
     439     return

File /usr/lib/python3.14/asyncio/runners.py:201, in run(main, debug, loop_fac
199 if events._get_running_loop() is not None:
200     # fail fast with short traceback
--> 201     raise RuntimeError(
202         "asyncio.run() cannot be called from a running event loop")
204 with Runner(debug=debug, loop_factory=loop_factory) as runner:

RuntimeError: asyncio.run() cannot be called from a running event loop

During handling of the above exception, another exception occurred:

AttributeError                                Traceback (most recent call last)
Cell In[4], line 64
     60     try:
     61         main() # main function which runs in an internal infinite lo
     62     except:
     63         exc_type, exc_value, exc_traceback = sys.exc_info()
--> 64         traceback.print_tb(exc_traceback, file=print)

File /usr/lib/python3.14/traceback.py:63, in print_tb(tb, limit, file)
     55 def print_tb(tb, limit=None, file=None):
     56     """Print up to 'limit' stack trace entries from the traceback 'tb'
     57
     58     If 'limit' is omitted or None, all entries are printed. If 'file'
(...)     61     method.
     62     """
--> 63     print_list(extract_tb(tb, limit=limit), file=file)

File /usr/lib/python3.14/traceback.py:82, in extract_tb(tb, limit)
     69 def extract_tb(tb, limit=None):
     70     """
     71     Return a StackSummary object representing a list of
     72     pre-processed entries from traceback.
(...)     80     available the corresponding attribute is None.
     81     """
--> 82     return StackSummary._extract_from_extended_frame_gen(
     83         _walk_tb_with_full_positions(tb), limit=limit)

File /usr/lib/python3.14/traceback.py:481, in StackSummary._extract_from_exte
rame_gen(klass, frame_gen, limit, lookup_lines, capture_locals)
     479 result = klass()

```

```
480 fnames = set()
--> 481 for f, (lineno, end_lineno, colno, end_colno) in frame_gen:
482     co = f.f_code
483     filename = co.co_filename

File /usr/lib/python3.14/traceback.py:413, in _walk_tb_with_full_positions(tb)
409 def _walk_tb_with_full_positions(tb):
410     # Internal version of walk_tb that yields full code positions inc
411     # end line and column information.
412     while tb is not None:
--> 413         positions = _get_code_position(tb.tb_frame.f_code, tb.tb_last
414         # Yield tb_lineno when co_positions does not have a line numb
415         # maintain behavior with walk_tb.
416         if positions[0] is None:

AttributeError: type object 'RuntimeError' has no attribute 'tb_frame'
```

In []: